Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Практическая работа №3
По дисциплине «СПП»
Вариант 14

Выполнил:
Студент 3 курса
Группы ПО-12
Мельник Р.С
Проверил:
Кулик А.Д.

Брест 2026

Цель работы:закрепить базовые знания языка программирования Python при решении практических задач

1) 3) Проект «Бургер-закусочная». Реализовать возможность формирования заказа из определенных позиций (тип бургера (веганский, куриный и т.д.)), напиток (холодный – пепси, кока-кола и т.д.; горячий – кофе, чай и т.д.), тип упаковки – с собой, на месте. Должна формироваться итоговая стоимость заказа.

2) Проект «IT-компания». В проекте должен быть реализован класс «Сотрудник» с субординацией (т.е. должна быть возможность определения кому подчиняется сотрудник и кто находится в его подчинении). Для каждого сотрудника помимо сведений о субординации хранятся другие данные (ФИО, отдел, должность, зарплата). Предусмотреть возможность удаления и добавления сотрудника.

```
import os
import sys
from abc import ABC, abstractmethod
from enum import Enum
from typing import Optional, List, Dict, Any
import time


# ==================== ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ ====================


def clear_screen():
    """Очистка экрана (работает в Windows и Unix)"""
    os.system('cls' if os.name == 'nt' else 'clear')
```

```
def print_header(title):
    """Вывод заголовка с оформлением"""
    print("=" * 60)
    print(f"{title:^60}")
    print("=" * 60)


def print_menu_item(number, text):
    """Вывод пункта меню"""
    print(f"  {number}. {text}")


def wait_for_enter():
    """Ожидание нажатия Enter"""
    input("\nНажмите Enter для продолжения...")


# ==================== ЛАБОРАТОРНАЯ 1: БУРГЕР-ЗАКУСОЧНАЯ
====================

class BurgerType(Enum):
    VEGAN = "Веганский бургер"
    CHICKEN = "Куриный бургер"
    BEEF = "Говяжий бургер"
    FISH = "Рыбный бургер"


class ColdDrinkType(Enum):
    PEPSI = "Пепси"
    COCA_COLA = "Кока-кола"
```

```
    SPRITE = "Спрайт"
    FANTA = "Фанта"


class HotDrinkType(Enum):
    COFFEE = "Кофе"
    TEA = "Чай"
    HOT_CHOCOLATE = "Горячий шоколад"
    CAPPUCCINO = "Капучино"


class PackagingType(Enum):
    TAKEAWAY = "С собой"
    HERE = "На месте"


class Burger:
    def __init__(self, burger_type: BurgerType):
        self.burger_type = burger_type
        self.price = self._get_price()

    def _get_price(self) -> float:
        prices = {
            BurgerType.VEGAN: 250,
            BurgerType.CHICKEN: 280,
            BurgerType.BEEF: 320,
            BurgerType.FISH: 300
        }
        return prices.get(self.burger_type, 0)
```

```
    def __str__(self):
        return f"{self.burger_type.value} - {self.price} руб."


class Drink(ABC):
    def __init__(self):
        self.name = ""
        self.price = 0

    @abstractmethod
    def __str__(self):
        pass


class ColdDrink(Drink):
    def __init__(self, drink_type: ColdDrinkType):
        super().__init__()
        self.drink_type = drink_type
        self.price = self._get_price()

    def _get_price(self) -> float:
        prices = {
            ColdDrinkType.PEPSI: 80,
            ColdDrinkType.COCA_COLA: 85,
            ColdDrinkType.SPRITE: 85,
            ColdDrinkType.FANTA: 85
        }
        return prices.get(self.drink_type, 0)

    def __str__(self):
```

```
        return f"Холодный напиток: {self.drink_type.value} - {self.price} руб."


class HotDrink(Drink):
    def __init__(self, drink_type: HotDrinkType):
        super().__init__()
        self.drink_type = drink_type
        self.price = self._get_price()

    def _get_price(self) -> float:
        prices = {
            HotDrinkType.COFFEE: 150,
            HotDrinkType.TEA: 100,
            HotDrinkType.HOT_CHOCOLATE: 180,
            HotDrinkType.CAPPUCCINO: 200
        }
        return prices.get(self.drink_type, 0)

    def __str__(self):
        return f"Горячий напиток: {self.drink_type.value} - {self.price} руб."


class Packaging:
    def __init__(self, packaging_type: PackagingType):
        self.packaging_type = packaging_type
        self.price = 0 if packaging_type == PackagingType.HERE else 20

    def __str__(self):
        return f"Упаковка: {self.packaging_type.value} - {self.price} руб."
```

```
class Order:
    def __init__(self):
        self.burger: Optional[Burger] = None
        self.drink: Optional[Drink] = None
        self.packaging: Optional[Packaging] = None

    def calculate_total(self) -> float:
        total = 0
        if self.burger:
            total += self.burger.price
        if self.drink:
            total += self.drink.price
        if self.packaging:
            total += self.packaging.price
        return total

    def is_complete(self):
        return self.burger is not None

    def __str__(self):
        order_details = ["\n" + "=" * 40, "ВАШ ЗАКАЗ".center(40), "=" * 40]
        if self.burger:
            order_details.append(f"🍔 {str(self.burger)}")
        if self.drink:
            order_details.append(f"🥤 {str(self.drink)}")
        if self.packaging:
            order_details.append(f"📦 {str(self.packaging)}")

        order_details.append("-" * 40)
```

```
        order_details.append(f"ИТОГО: {self.calculate_total()} руб.".center(40))
        order_details.append("=" * 40)
        return "\n".join(order_details)


class OrderBuilder(ABC):
    @abstractmethod
    def reset(self):
        pass

    @abstractmethod
    def add_burger(self, burger_type: BurgerType) -> 'OrderBuilder':
        pass

    @abstractmethod
    def add_cold_drink(self, drink_type: ColdDrinkType) -> 'OrderBuilder':
        pass

    @abstractmethod
    def add_hot_drink(self, drink_type: HotDrinkType) -> 'OrderBuilder':
        pass

    @abstractmethod
    def set_packaging(self, packaging_type: PackagingType) -> 'OrderBuilder':
        pass

    @abstractmethod
    def build(self) -> Order:
        pass
```

```
class ConcreteOrderBuilder(OrderBuilder):
    def __init__(self):
        self.reset()

    def reset(self):
        self.order = Order()
        return self

    def add_burger(self, burger_type: BurgerType) -> 'OrderBuilder':
        self.order.burger = Burger(burger_type)
        return self

    def add_cold_drink(self, drink_type: ColdDrinkType) -> 'OrderBuilder':
        self.order.drink = ColdDrink(drink_type)
        return self

    def add_hot_drink(self, drink_type: HotDrinkType) -> 'OrderBuilder':
        self.order.drink = HotDrink(drink_type)
        return self

    def set_packaging(self, packaging_type: PackagingType) -> 'OrderBuilder':
        self.order.packaging = Packaging(packaging_type)
        return self

    def build(self) -> Order:
        built_order = self.order
        self.reset()
        return built_order
```

```
class BurgerApp:
    def __init__(self):
        self.builder = ConcreteOrderBuilder()
        self.orders_history = []

    def show_menu(self):
        """Показать меню бургерной"""
        clear_screen()
        print_header("🍔 БУРГЕР-ЗАКУСОЧНАЯ 🍔")

        print("\n1. Сделать заказ")
        print("2. Показать историю заказов")
        print("3. Вернуться в главное меню")

        choice = input("\nВыберите действие: ")
        return choice

    def select_burger(self):
        """Выбор бургера"""
        clear_screen()
        print_header("ВЫБОР БУРГЕРА")

        burgers = list(BurgerType)
        for i, burger in enumerate(burgers, 1):
            price = Burger(burger).price
            print(f"{i}. {burger.value} - {price} руб.")

        print(f"{len(burgers) + 1}. Отмена")
```

```
    while True:
        try:
            choice = int(input("\nВыберите бургер: "))
            if 1 <= choice <= len(burgers):
                return burgers[choice - 1]
            elif choice == len(burgers) + 1:
                return None
            else:
                print("Неверный выбор!")
        except ValueError:
            print("Введите число!")

def select_drink(self):
    """Выбор напитка"""
    clear_screen()
    print_header("ВЫБОР НАПИТКА")

    print("1. Холодные напитки")
    print("2. Горячие напитки")
    print("3. Без напитка")
    print("4. Отмена")

    choice = input("\nВыберите тип напитка: ")

    if choice == '1':
        return self.select_cold_drink()
    elif choice == '2':
        return self.select_hot_drink()
    elif choice == '3':
        return "none"
```

```
    elif choice == '4':
        return None
    else:
        print("Неверный выбор!")
        return self.select_drink()

def select_cold_drink(self):
    """Выбор холодного напитка"""
    clear_screen()
    print_header("ХОЛОДНЫЕ НАПИТКИ")

    drinks = list(ColdDrinkType)
    for i, drink in enumerate(drinks, 1):
        price = ColdDrink(drink).price
        print(f"{i}. {drink.value} - {price} руб.")

    print(f"{len(drinks) + 1}. Назад")

    while True:
        try:
            choice = int(input("\nВыберите напиток: "))
            if 1 <= choice <= len(drinks):
                return drinks[choice - 1]
            elif choice == len(drinks) + 1:
                return self.select_drink()
            else:
                print("Неверный выбор!")
        except ValueError:
            print("Введите число!")
```

```
def select_hot_drink(self):
    """Выбор горячего напитка"""
    clear_screen()
    print_header("ГОРЯЧИЕ НАПИТКИ")

    drinks = list(HotDrinkType)
    for i, drink in enumerate(drinks, 1):
        price = HotDrink(drink).price
        print(f"{i}. {drink.value} - {price} руб.")

    print(f"{len(drinks) + 1}. Назад")

    while True:
        try:
            choice = int(input("\nВыберите напиток: "))
            if 1 <= choice <= len(drinks):
                return drinks[choice - 1]
            elif choice == len(drinks) + 1:
                return self.select_drink()
            else:
                print("Неверный выбор!")
        except ValueError:
            print("Введите число!")

def select_packaging(self):
    """Выбор упаковки"""
    clear_screen()
    print_header("ВЫБОР УПАКОВКИ")

    packaging_types = list(PackagingType)
```

```
    for i, pkg in enumerate(packaging_types, 1):
        price = 0 if pkg == PackagingType.HERE else 20
        print(f"{i}. {pkg.value} - {price} руб.")

    print(f"{len(packaging_types) + 1}. Отмена")

    while True:
        try:
            choice = int(input("\nВыберите тип упаковки: "))
            if 1 <= choice <= len(packaging_types):
                return packaging_types[choice - 1]
            elif choice == len(packaging_types) + 1:
                return None
            else:
                print("Неверный выбор!")
        except ValueError:
            print("Введите число!")

def make_order(self):
    """Процесс создания заказа"""
    clear_screen()
    print_header("🍔 СОЗДАНИЕ ЗАКАЗА 🍔")

    # Сброс билдера
    self.builder.reset()

    # Выбор бургера
    burger = self.select_burger()
    if not burger:
        print("\nЗаказ отменен.")
```

```
    wait_for_enter()
    return

self.builder.add_burger(burger)

# Выбор напитка
drink_result = self.select_drink()
if drink_result is None:
    print("\nЗаказ отменен.")
    wait_for_enter()
    return
elif drink_result != "none":
    if isinstance(drink_result, ColdDrinkType):
        self.builder.add_cold_drink(drink_result)
    else:
        self.builder.add_hot_drink(drink_result)

# Выбор упаковки
packaging = self.select_packaging()
if packaging is None:
    print("\nЗаказ отменен.")
    wait_for_enter()
    return

self.builder.set_packaging(packaging)

# Создание заказа
order = self.builder.build()
self.orders_history.append(order)
```

```
    # Показ результата
    clear_screen()
    print(order)

    wait_for_enter()

def show_history(self):
    """Показать историю заказов"""
    clear_screen()
    print_header("📋 ИСТОРИЯ ЗАКАЗОВ")

    if not self.orders_history:
        print("\nИстория заказов пуста.")
    else:
        for i, order in enumerate(self.orders_history, 1):
            print(f"\n--- Заказ #{i} ---")
            print(f"Сумма: {order.calculate_total()} руб.")
            if order.burger:
                print(f"  {order.burger}")
            if order.drink:
                print(f"  {order.drink}")
            if order.packaging:
                print(f"  {order.packaging}")

    wait_for_enter()

def run(self):
    """Запуск приложения бургерной"""
    while True:
        choice = self.show_menu()
```

```
        if choice == '1':
            self.make_order()
        elif choice == '2':
            self.show_history()
        elif choice == '3':
            break
        else:
            print("\nНеверный выбор!")
            wait_for_enter()


# ==================== ЛАБОРАТОРНАЯ 2: IT-КОМПАНИЯ ====================

class EmployeeComponent(ABC):
    @abstractmethod
    def get_info(self, level: int = 0) -> str:
        pass

    @abstractmethod
    def get_salary(self) -> float:
        pass

    @abstractmethod
    def find_employee(self, full_name: str) -> Optional['EmployeeComponent']:
        pass

    @abstractmethod
    def get_full_name(self) -> str:
        pass
```

```
class Employee(EmployeeComponent):
    def __init__(self, full_name: str, department: str, position: str, salary: float):
        self.full_name = full_name
        self.department = department
        self.position = position
        self.salary = salary
        self.manager: Optional[EmployeeComponent] = None

    def set_manager(self, manager: 'EmployeeComponent'):
        self.manager = manager

    def get_full_name(self) -> str:
        return self.full_name

    def get_info(self, level: int = 0) -> str:
        indent = "  " * level
        info = f"{indent}👤 {self.full_name} - {self.position} ({self.department}) - {self.salary} руб."
        return info

    def get_salary(self) -> float:
        return self.salary

    def find_employee(self, full_name: str) -> Optional['EmployeeComponent']:
        if self.full_name.lower() == full_name.lower():
            return self
        return None
```

```
class Manager(Employee):
    def __init__(self, full_name: str, department: str, position: str, salary: float):
        super().__init__(full_name, department, position, salary)
        self.subordinates: List[EmployeeComponent] = []

    def add_subordinate(self, employee: EmployeeComponent):
        self.subordinates.append(employee)
        if isinstance(employee, Employee):
            employee.set_manager(self)

    def remove_subordinate(self, employee: EmployeeComponent):
        if employee in self.subordinates:
            self.subordinates.remove(employee)
            if isinstance(employee, Employee):
                employee.set_manager(None)

    def get_subordinates(self) -> List[EmployeeComponent]:
        return self.subordinates.copy()

    def get_info(self, level: int = 0) -> str:
        indent = "  " * level
        info = [f"{indent}👔 {self.full_name} - {self.position} ({self.department}) - {self.salary} руб. (Руководитель)"]

        if self.subordinates:
            info.append(f"{indent}  Подчиненные:")
            for subordinate in self.subordinates:
                info.append(subordinate.get_info(level + 2))

        return "\n".join(info)
```

```
    def get_salary(self) -> float:
        total = self.salary
        for subordinate in self.subordinates:
            total += subordinate.get_salary()
        return total

    def find_employee(self, full_name: str) -> Optional['EmployeeComponent']:
        if self.full_name.lower() == full_name.lower():
            return self

        for subordinate in self.subordinates:
            result = subordinate.find_employee(full_name)
            if result:
                return result
        return None


class Company:
    def __init__(self, name: str = "ТехноИнновации"):
        self.name = name
        self.ceo: Optional[Manager] = None
        self.all_employees: List[EmployeeComponent] = []

    def set_ceo(self, ceo: Manager):
        self.ceo = ceo
        self.add_employee(ceo)

    def add_employee(self, employee: EmployeeComponent):
        if employee not in self.all_employees:
            self.all_employees.append(employee)
```

```
def remove_employee(self, employee: EmployeeComponent):
    if employee in self.all_employees:
        if isinstance(employee, Employee) and employee.manager:
            if isinstance(employee.manager, Manager):
                employee.manager.remove_subordinate(employee)

        if isinstance(employee, Manager):
            for subordinate in employee.get_subordinates():
                if employee.manager and isinstance(employee.manager, Manager):
                    employee.manager.add_subordinate(subordinate)
                elif self.ceo and employee != self.ceo:
                    self.ceo.add_subordinate(subordinate)

        self.all_employees.remove(employee)

def find_employee(self, full_name: str) -> Optional[EmployeeComponent]:
    if self.ceo:
        return self.ceo.find_employee(full_name)
    return None

def get_total_salary(self) -> float:
    if self.ceo:
        return self.ceo.get_salary()
    return 0

def get_company_structure(self) -> str:
    if not self.ceo:
        return "В компании нет сотрудников"
```

```
        result = [f"\n{'=' * 50}", f"{self.name}".center(50), '=' * 50]
        result.append("Организационная структура:")
        result.append(self.ceo.get_info())
        result.append('-' * 50)
        result.append(f"Общий фонд оплаты труда: {self.get_total_salary()} руб.".center(50))
        result.append('=' * 50)
        return "\n".join(result)

    def get_all_employees_list(self) -> str:
        if not self.all_employees:
            return "Нет сотрудников"

        result = ["\n" + "=" * 50, "СПИСОК ВСЕХ СОТРУДНИКОВ".center(50), "=" * 50]
        for emp in self.all_employees:
            result.append(f"  • {emp.get_full_name()} - {emp.position}")
        result.append("=" * 50)
        return "\n".join(result)


class ITCompanyApp:
    def __init__(self):
        self.company = Company()
        self.init_default_company()

    def init_default_company(self):
        """Инициализация компании с тестовыми данными"""
        # Создаем руководство
        ceo = Manager("Иванов Иван Иванович", "Руководство", "Генеральный директор",
500000)
        self.company.set_ceo(ceo)
```

```
# Создаем руководителей отделов
tech_director = Manager("Петров Петр Петрович", "IT-департамент", "Технический директор", 300000)
hr_director = Manager("Сидорова Анна Сергеевна", "HR-департамент", "Директор по персоналу", 250000)
sales_director = Manager("Козлов Дмитрий Андреевич", "Отдел продаж", "Руководитель отдела продаж", 280000)

ceo.add_subordinate(tech_director)
ceo.add_subordinate(hr_director)
ceo.add_subordinate(sales_director)

self.company.add_employee(tech_director)
self.company.add_employee(hr_director)
self.company.add_employee(sales_director)

# IT отдел
dev1 = Employee("Алексеев Алексей Алексеевич", "IT-департамент", "Senior Developer", 180000)
dev2 = Employee("Михайлов Михаил Михайлович", "IT-департамент", "Middle Developer", 120000)
dev3 = Employee("Николаев Николай Николаевич", "IT-департамент", "Junior Developer", 80000)

tech_director.add_subordinate(dev1)
tech_director.add_subordinate(dev2)
tech_director.add_subordinate(dev3)

self.company.add_employee(dev1)
self.company.add_employee(dev2)
self.company.add_employee(dev3)
```

```
        # HR отдел
        hr1 = Employee("Васильева Елена Владимировна", "HR-департамент", "HR-менеджер", 90000)
        hr2 = Employee("Павлова Ольга Игоревна", "HR-департамент", "Рекрутер", 70000)

        hr_director.add_subordinate(hr1)
        hr_director.add_subordinate(hr2)

        self.company.add_employee(hr1)
        self.company.add_employee(hr2)

        # Отдел продаж
        sales1 = Employee("Соколов Артем Викторович", "Отдел продаж", "Менеджер по продажам", 100000)
        sales2 = Employee("Федорова Татьяна Дмитриевна", "Отдел продаж", "Менеджер по продажам", 100000)

        sales_director.add_subordinate(sales1)
        sales_director.add_subordinate(sales2)

        self.company.add_employee(sales1)
        self.company.add_employee(sales2)

    def show_menu(self):
        """Показать меню IT-компании"""
        clear_screen()
        print_header("💻 IT-КОМПАНИЯ 💻")

        print("\n1. Показать структуру компании")
        print("2. Показать всех сотрудников")
        print("3. Найти сотрудника")
```

```
    print("4. Добавить сотрудника")
    print("5. Удалить сотрудника")
    print("6. Вернуться в главное меню")

    choice = input("\nВыберите действие: ")
    return choice

def find_employee_ui(self):
    """Поиск сотрудника"""
    clear_screen()
    print_header("🔍 ПОИСК СОТРУДНИКА")

    name = input("Введите ФИО сотрудника: ")
    employee = self.company.find_employee(name)

    if employee:
        print(f"\nНайден сотрудник:")
        print(employee.get_info())
    else:
        print(f"\nСотрудник с ФИО '{name}' не найден.")

    wait_for_enter()

def add_employee_ui(self):
    """Добавление сотрудника"""
    clear_screen()
    print_header("➕ ДОБАВЛЕНИЕ СОТРУДНИКА")

    print("Тип сотрудника:")
    print("1. Обычный сотрудник")
```

```
    print("2. Руководитель")
    print("3. Отмена")

    emp_type = input("\nВыберите тип: ")

    if emp_type == '3':
        return

    # Ввод общих данных
    full_name = input("Введите ФИО: ")
    department = input("Введите отдел: ")
    position = input("Введите должность: ")

    try:
        salary = float(input("Введите зарплату: "))
    except ValueError:
        print("Ошибка: введите число!")
        wait_for_enter()
        return

    # Выбор менеджера
    print("\nВведите ФИО руководителя (или оставьте пустым, если руководитель не
указан):")
    manager_name = input()

    if emp_type == '1':
        new_employee = Employee(full_name, department, position, salary)
    else:
        new_employee = Manager(full_name, department, position, salary)

    # Добавление в структуру
```

```
    if manager_name:
        manager = self.company.find_employee(manager_name)
        if manager and isinstance(manager, Manager):
            manager.add_subordinate(new_employee)
        else:
            print("Руководитель не найден или не является менеджером!")
            wait_for_enter()
            return
    elif self.company.ceo:
        self.company.ceo.add_subordinate(new_employee)

    self.company.add_employee(new_employee)
    print(f"\nСотрудник {full_name} успешно добавлен!")
    wait_for_enter()

def remove_employee_ui(self):
    """Удаление сотрудника"""
    clear_screen()
    print_header("➖ УДАЛЕНИЕ СОТРУДНИКА")

    name = input("Введите ФИО сотрудника для удаления: ")
    employee = self.company.find_employee(name)

    if not employee:
        print(f"\nСотрудник с ФИО '{name}' не найден.")
        wait_for_enter()
        return

    print(f"\nНайден сотрудник:")
    print(employee.get_info())
```

```
        print()

        confirm = input("Вы уверены, что хотите удалить этого сотрудника? (д/н): ")
        if confirm.lower() in ['д', 'да', 'y', 'yes']:
            self.company.remove_employee(employee)
            print("Сотрудник удален!")
        else:
            print("Удаление отменено.")

        wait_for_enter()

    def run(self):
        """Запуск приложения IT-компании"""
        while True:
            choice = self.show_menu()

            if choice == '1':
                clear_screen()
                print(self.company.get_company_structure())
                wait_for_enter()
            elif choice == '2':
                clear_screen()
                print(self.company.get_all_employees_list())
                wait_for_enter()
            elif choice == '3':
                self.find_employee_ui()
            elif choice == '4':
                self.add_employee_ui()
            elif choice == '5':
                self.remove_employee_ui()
```

```
        elif choice == '6':
            break
        else:
            print("\nНеверный выбор!")
            wait_for_enter()


# ==================== ГЛАВНОЕ МЕНЮ ====================

class MainApp:
    def __init__(self):
        self.burger_app = BurgerApp()
        self.it_company_app = ITCompanyApp()

    def show_main_menu(self):
        """Показать главное меню"""
        clear_screen()
        print_header="""
    ╔═══════════════════════════════════════════════════════╗
    ║    ЛАБОРАТОРНЫЕ РАБОТЫ ПО ПАТТЕРНАМ          ║
    ║       ПРОЕКТИРОВАНИЯ                          ║
    ╚═══════════════════════════════════════════════════════╝
    """

        print("\nДоступные лабораторные работы:\n")
        print_menu_item(1, "🍔 Бургер-закусочная (Порождающий паттерн - Builder)")
        print_menu_item(2, "💻 IT-компания (Структурный паттерн - Composite)")
        print_menu_item(0, "🚪 Выход")

        choice = input("\nВыберите лабораторную работу: ")
```

```
        return choice

    def run(self):
        """Запуск главного приложения"""
        while True:
            choice = self.show_main_menu()

            if choice == '1':
                self.burger_app.run()
            elif choice == '2':
                self.it_company_app.run()
            elif choice == '0':
                clear_screen()
                print_header("ДО СВИДАНИЯ!")
                print("\nСпасибо за использование программы!")
                time.sleep(2)
                sys.exit(0)
            else:
                print("\nНеверный выбор!")
                wait_for_enter()


# ==================== ТОЧКА ВХОДА ====================

if __name__ == "__main__":
    app = MainApp()
    app.run()#!/usr/bin/env python3
# -*- coding: utf-8 -*-

"""
```

```
Лабораторные работы по паттернам проектирования
Объединенное приложение с меню выбора
"""

import os
import sys
from abc import ABC, abstractmethod
from enum import Enum
from typing import Optional, List, Dict, Any
import time


# ==================== ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ ====================

def clear_screen():
    """Очистка экрана (работает в Windows и Unix)"""
    os.system('cls' if os.name == 'nt' else 'clear')


def print_header(title):
    """Вывод заголовка с оформлением"""
    print("=" * 60)
    print(f"{title:^60}")
    print("=" * 60)


def print_menu_item(number, text):
    """Вывод пункта меню"""
    print(f"  {number}. {text}")
```

```
def wait_for_enter():
    """Ожидание нажатия Enter"""
    input("\nНажмите Enter для продолжения...")


# ==================== ЛАБОРАТОРНАЯ 1: БУРГЕР-ЗАКУСОЧНАЯ ====================

class BurgerType(Enum):
    VEGAN = "Веганский бургер"
    CHICKEN = "Куриный бургер"
    BEEF = "Говяжий бургер"
    FISH = "Рыбный бургер"


class ColdDrinkType(Enum):
    PEPSI = "Пепси"
    COCA_COLA = "Кока-кола"
    SPRITE = "Спрайт"
    FANTA = "Фанта"


class HotDrinkType(Enum):
    COFFEE = "Кофе"
    TEA = "Чай"
    HOT_CHOCOLATE = "Горячий шоколад"
    CAPPUCCINO = "Капучино"


class PackagingType(Enum):
```

```
    TAKEAWAY = "С собой"
    HERE = "На месте"


class Burger:
    def __init__(self, burger_type: BurgerType):
        self.burger_type = burger_type
        self.price = self._get_price()

    def _get_price(self) -> float:
        prices = {
            BurgerType.VEGAN: 250,
            BurgerType.CHICKEN: 280,
            BurgerType.BEEF: 320,
            BurgerType.FISH: 300
        }
        return prices.get(self.burger_type, 0)

    def __str__(self):
        return f"{self.burger_type.value} - {self.price} руб."


class Drink(ABC):
    def __init__(self):
        self.name = ""
        self.price = 0

    @abstractmethod
    def __str__(self):
        pass
```

```
class ColdDrink(Drink):
    def __init__(self, drink_type: ColdDrinkType):
        super().__init__()
        self.drink_type = drink_type
        self.price = self._get_price()

    def _get_price(self) -> float:
        prices = {
            ColdDrinkType.PEPSI: 80,
            ColdDrinkType.COCA_COLA: 85,
            ColdDrinkType.SPRITE: 85,
            ColdDrinkType.FANTA: 85
        }
        return prices.get(self.drink_type, 0)

    def __str__(self):
        return f"Холодный напиток: {self.drink_type.value} - {self.price} руб."


class HotDrink(Drink):
    def __init__(self, drink_type: HotDrinkType):
        super().__init__()
        self.drink_type = drink_type
        self.price = self._get_price()

    def _get_price(self) -> float:
        prices = {
            HotDrinkType.COFFEE: 150,
```

```
            HotDrinkType.TEA: 100,
            HotDrinkType.HOT_CHOCOLATE: 180,
            HotDrinkType.CAPPUCCINO: 200
        }
        return prices.get(self.drink_type, 0)

    def __str__(self):
        return f"Горячий напиток: {self.drink_type.value} - {self.price} руб."


class Packaging:
    def __init__(self, packaging_type: PackagingType):
        self.packaging_type = packaging_type
        self.price = 0 if packaging_type == PackagingType.HERE else 20

    def __str__(self):
        return f"Упаковка: {self.packaging_type.value} - {self.price} руб."


class Order:
    def __init__(self):
        self.burger: Optional[Burger] = None
        self.drink: Optional[Drink] = None
        self.packaging: Optional[Packaging] = None

    def calculate_total(self) -> float:
        total = 0
        if self.burger:
            total += self.burger.price
        if self.drink:
```

```
            total += self.drink.price
        if self.packaging:
            total += self.packaging.price
        return total

    def is_complete(self):
        return self.burger is not None

    def __str__(self):
        order_details = ["\n" + "=" * 40, "ВАШ ЗАКАЗ".center(40), "=" * 40]
        if self.burger:
            order_details.append(f"🍔 {str(self.burger)}")
        if self.drink:
            order_details.append(f"🥤 {str(self.drink)}")
        if self.packaging:
            order_details.append(f"📦 {str(self.packaging)}")

        order_details.append("-" * 40)
        order_details.append(f"ИТОГО: {self.calculate_total()} руб.".center(40))
        order_details.append("=" * 40)
        return "\n".join(order_details)


class OrderBuilder(ABC):
    @abstractmethod
    def reset(self):
        pass

    @abstractmethod
    def add_burger(self, burger_type: BurgerType) -> 'OrderBuilder':
```

```
        pass

    @abstractmethod
    def add_cold_drink(self, drink_type: ColdDrinkType) -> 'OrderBuilder':
        pass

    @abstractmethod
    def add_hot_drink(self, drink_type: HotDrinkType) -> 'OrderBuilder':
        pass

    @abstractmethod
    def set_packaging(self, packaging_type: PackagingType) -> 'OrderBuilder':
        pass

    @abstractmethod
    def build(self) -> Order:
        pass


class ConcreteOrderBuilder(OrderBuilder):
    def __init__(self):
        self.reset()

    def reset(self):
        self.order = Order()
        return self

    def add_burger(self, burger_type: BurgerType) -> 'OrderBuilder':
        self.order.burger = Burger(burger_type)
        return self
```

```
    def add_cold_drink(self, drink_type: ColdDrinkType) -> 'OrderBuilder':
        self.order.drink = ColdDrink(drink_type)
        return self

    def add_hot_drink(self, drink_type: HotDrinkType) -> 'OrderBuilder':
        self.order.drink = HotDrink(drink_type)
        return self

    def set_packaging(self, packaging_type: PackagingType) -> 'OrderBuilder':
        self.order.packaging = Packaging(packaging_type)
        return self

    def build(self) -> Order:
        built_order = self.order
        self.reset()
        return built_order


class BurgerApp:
    def __init__(self):
        self.builder = ConcreteOrderBuilder()
        self.orders_history = []

    def show_menu(self):
        """Показать меню бургерной"""
        clear_screen()
        print_header("🍔 БУРГЕР-ЗАКУСОЧНАЯ 🍔")

        print("\n1. Сделать заказ")
```

```
    print("2. Показать историю заказов")
    print("3. Вернуться в главное меню")

    choice = input("\nВыберите действие: ")
    return choice

def select_burger(self):
    """Выбор бургера"""
    clear_screen()
    print_header("ВЫБОР БУРГЕРА")

    burgers = list(BurgerType)
    for i, burger in enumerate(burgers, 1):
        price = Burger(burger).price
        print(f"{i}. {burger.value} - {price} руб.")

    print(f"{len(burgers) + 1}. Отмена")

    while True:
        try:
            choice = int(input("\nВыберите бургер: "))
            if 1 <= choice <= len(burgers):
                return burgers[choice - 1]
            elif choice == len(burgers) + 1:
                return None
            else:
                print("Неверный выбор!")
        except ValueError:
            print("Введите число!")
```

```
def select_drink(self):
    """Выбор напитка"""
    clear_screen()
    print_header("ВЫБОР НАПИТКА")

    print("1. Холодные напитки")
    print("2. Горячие напитки")
    print("3. Без напитка")
    print("4. Отмена")

    choice = input("\nВыберите тип напитка: ")

    if choice == '1':
        return self.select_cold_drink()
    elif choice == '2':
        return self.select_hot_drink()
    elif choice == '3':
        return "none"
    elif choice == '4':
        return None
    else:
        print("Неверный выбор!")
        return self.select_drink()

def select_cold_drink(self):
    """Выбор холодного напитка"""
    clear_screen()
    print_header("ХОЛОДНЫЕ НАПИТКИ")

    drinks = list(ColdDrinkType)
```

```
    for i, drink in enumerate(drinks, 1):
        price = ColdDrink(drink).price
        print(f"{i}. {drink.value} - {price} руб.")

    print(f"{len(drinks) + 1}. Назад")

    while True:
        try:
            choice = int(input("\nВыберите напиток: "))
            if 1 <= choice <= len(drinks):
                return drinks[choice - 1]
            elif choice == len(drinks) + 1:
                return self.select_drink()
            else:
                print("Неверный выбор!")
        except ValueError:
            print("Введите число!")

def select_hot_drink(self):
    """Выбор горячего напитка"""
    clear_screen()
    print_header("ГОРЯЧИЕ НАПИТКИ")

    drinks = list(HotDrinkType)
    for i, drink in enumerate(drinks, 1):
        price = HotDrink(drink).price
        print(f"{i}. {drink.value} - {price} руб.")

    print(f"{len(drinks) + 1}. Назад")
```

```
    while True:
        try:
            choice = int(input("\nВыберите напиток: "))
            if 1 <= choice <= len(drinks):
                return drinks[choice - 1]
            elif choice == len(drinks) + 1:
                return self.select_drink()
            else:
                print("Неверный выбор!")
        except ValueError:
            print("Введите число!")

def select_packaging(self):
    """Выбор упаковки"""
    clear_screen()
    print_header("ВЫБОР УПАКОВКИ")

    packaging_types = list(PackagingType)
    for i, pkg in enumerate(packaging_types, 1):
        price = 0 if pkg == PackagingType.HERE else 20
        print(f"{i}. {pkg.value} - {price} руб.")

    print(f"{len(packaging_types) + 1}. Отмена")

    while True:
        try:
            choice = int(input("\nВыберите тип упаковки: "))
            if 1 <= choice <= len(packaging_types):
                return packaging_types[choice - 1]
            elif choice == len(packaging_types) + 1:
```

```
            return None
        else:
            print("Неверный выбор!")
    except ValueError:
        print("Введите число!")

def make_order(self):
    """Процесс создания заказа"""
    clear_screen()
    print_header("🍔 СОЗДАНИЕ ЗАКАЗА 🍔")

    # Сброс билдера
    self.builder.reset()

    # Выбор бургера
    burger = self.select_burger()
    if not burger:
        print("\nЗаказ отменен.")
        wait_for_enter()
        return

    self.builder.add_burger(burger)

    # Выбор напитка
    drink_result = self.select_drink()
    if drink_result is None:
        print("\nЗаказ отменен.")
        wait_for_enter()
        return
    elif drink_result != "none":
```

```
        if isinstance(drink_result, ColdDrinkType):
            self.builder.add_cold_drink(drink_result)
        else:
            self.builder.add_hot_drink(drink_result)

    # Выбор упаковки
    packaging = self.select_packaging()
    if packaging is None:
        print("\nЗаказ отменен.")
        wait_for_enter()
        return

    self.builder.set_packaging(packaging)

    # Создание заказа
    order = self.builder.build()
    self.orders_history.append(order)

    # Показ результата
    clear_screen()
    print(order)

    wait_for_enter()

def show_history(self):
    """Показать историю заказов"""
    clear_screen()
    print_header("📋 ИСТОРИЯ ЗАКАЗОВ")

    if not self.orders_history:
```

```
        print("\nИстория заказов пуста.")
    else:
        for i, order in enumerate(self.orders_history, 1):
            print(f"\n--- Заказ #{i} ---")
            print(f"Сумма: {order.calculate_total()} руб.")
            if order.burger:
                print(f"  {order.burger}")
            if order.drink:
                print(f"  {order.drink}")
            if order.packaging:
                print(f"  {order.packaging}")

    wait_for_enter()

def run(self):
    """Запуск приложения бургерной"""
    while True:
        choice = self.show_menu()

        if choice == '1':
            self.make_order()
        elif choice == '2':
            self.show_history()
        elif choice == '3':
            break
        else:
            print("\nНеверный выбор!")
            wait_for_enter()
```

```
# ==================== ЛАБОРАТОРНАЯ 2: IT-КОМПАНИЯ ====================

class EmployeeComponent(ABC):
    @abstractmethod
    def get_info(self, level: int = 0) -> str:
        pass

    @abstractmethod
    def get_salary(self) -> float:
        pass

    @abstractmethod
    def find_employee(self, full_name: str) -> Optional['EmployeeComponent']:
        pass

    @abstractmethod
    def get_full_name(self) -> str:
        pass


class Employee(EmployeeComponent):
    def __init__(self, full_name: str, department: str, position: str, salary: float):
        self.full_name = full_name
        self.department = department
        self.position = position
        self.salary = salary
        self.manager: Optional[EmployeeComponent] = None

    def set_manager(self, manager: 'EmployeeComponent'):
        self.manager = manager
```

```
    def get_full_name(self) -> str:
        return self.full_name

    def get_info(self, level: int = 0) -> str:
        indent = "  " * level
        info = f"{indent}👤 {self.full_name} - {self.position} ({self.department}) - {self.salary} руб."
        return info

    def get_salary(self) -> float:
        return self.salary

    def find_employee(self, full_name: str) -> Optional['EmployeeComponent']:
        if self.full_name.lower() == full_name.lower():
            return self
        return None


class Manager(Employee):
    def __init__(self, full_name: str, department: str, position: str, salary: float):
        super().__init__(full_name, department, position, salary)
        self.subordinates: List[EmployeeComponent] = []

    def add_subordinate(self, employee: EmployeeComponent):
        self.subordinates.append(employee)
        if isinstance(employee, Employee):
            employee.set_manager(self)

    def remove_subordinate(self, employee: EmployeeComponent):
        if employee in self.subordinates:
```

```
        self.subordinates.remove(employee)
        if isinstance(employee, Employee):
            employee.set_manager(None)

    def get_subordinates(self) -> List[EmployeeComponent]:
        return self.subordinates.copy()

    def get_info(self, level: int = 0) -> str:
        indent = "  " * level
        info = [f"{indent}👔 {self.full_name} - {self.position} ({self.department}) - {self.salary} руб. (Руководитель)"]

        if self.subordinates:
            info.append(f"{indent}  Подчиненные:")
            for subordinate in self.subordinates:
                info.append(subordinate.get_info(level + 2))

        return "\n".join(info)

    def get_salary(self) -> float:
        total = self.salary
        for subordinate in self.subordinates:
            total += subordinate.get_salary()
        return total

    def find_employee(self, full_name: str) -> Optional['EmployeeComponent']:
        if self.full_name.lower() == full_name.lower():
            return self

        for subordinate in self.subordinates:
            result = subordinate.find_employee(full_name)
```

```
        if result:
            return result
    return None


class Company:
    def __init__(self, name: str = "ТехноИнновации"):
        self.name = name
        self.ceo: Optional[Manager] = None
        self.all_employees: List[EmployeeComponent] = []

    def set_ceo(self, ceo: Manager):
        self.ceo = ceo
        self.add_employee(ceo)

    def add_employee(self, employee: EmployeeComponent):
        if employee not in self.all_employees:
            self.all_employees.append(employee)

    def remove_employee(self, employee: EmployeeComponent):
        if employee in self.all_employees:
            if isinstance(employee, Employee) and employee.manager:
                if isinstance(employee.manager, Manager):
                    employee.manager.remove_subordinate(employee)

            if isinstance(employee, Manager):
                for subordinate in employee.get_subordinates():
                    if employee.manager and isinstance(employee.manager, Manager):
                        employee.manager.add_subordinate(subordinate)
                    elif self.ceo and employee != self.ceo:
```

```
                self.ceo.add_subordinate(subordinate)

        self.all_employees.remove(employee)

    def find_employee(self, full_name: str) -> Optional[EmployeeComponent]:
        if self.ceo:
            return self.ceo.find_employee(full_name)
        return None

    def get_total_salary(self) -> float:
        if self.ceo:
            return self.ceo.get_salary()
        return 0

    def get_company_structure(self) -> str:
        if not self.ceo:
            return "В компании нет сотрудников"

        result = [f"\n{'=' * 50}", f"{self.name}".center(50), '=' * 50]
        result.append("Организационная структура:")
        result.append(self.ceo.get_info())
        result.append('-' * 50)
        result.append(f"Общий фонд оплаты труда: {self.get_total_salary()} руб.".center(50))
        result.append('=' * 50)
        return "\n".join(result)

    def get_all_employees_list(self) -> str:
        if not self.all_employees:
            return "Нет сотрудников"
```

```
        result = ["\n" + "=" * 50, "СПИСОК ВСЕХ СОТРУДНИКОВ".center(50), "=" * 50]
        for emp in self.all_employees:
            result.append(f"  • {emp.get_full_name()} - {emp.position}")
        result.append("=" * 50)
        return "\n".join(result)


class ITCompanyApp:
    def __init__(self):
        self.company = Company()
        self.init_default_company()

    def init_default_company(self):
        """Инициализация компании с тестовыми данными"""
        # Создаем руководство
        ceo = Manager("Иванов Иван Иванович", "Руководство", "Генеральный директор",
500000)
        self.company.set_ceo(ceo)

        # Создаем руководителей отделов
        tech_director = Manager("Петров Петр Петрович", "IT-департамент", "Технический
директор", 300000)
        hr_director = Manager("Сидорова Анна Сергеевна", "HR-департамент", "Директор по
персоналу", 250000)
        sales_director = Manager("Козлов Дмитрий Андреевич", "Отдел продаж",
"Руководитель отдела продаж", 280000)

        ceo.add_subordinate(tech_director)
        ceo.add_subordinate(hr_director)
        ceo.add_subordinate(sales_director)

        self.company.add_employee(tech_director)
```

```
        self.company.add_employee(hr_director)
        self.company.add_employee(sales_director)

        # IT отдел
        dev1 = Employee("Алексеев Алексей Алексеевич", "IT-департамент", "Senior Developer", 180000)
        dev2 = Employee("Михайлов Михаил Михайлович", "IT-департамент", "Middle Developer", 120000)
        dev3 = Employee("Николаев Николай Николаевич", "IT-департамент", "Junior Developer", 80000)

        tech_director.add_subordinate(dev1)
        tech_director.add_subordinate(dev2)
        tech_director.add_subordinate(dev3)

        self.company.add_employee(dev1)
        self.company.add_employee(dev2)
        self.company.add_employee(dev3)

        # HR отдел
        hr1 = Employee("Васильева Елена Владимировна", "HR-департамент", "HR-менеджер", 90000)
        hr2 = Employee("Павлова Ольга Игоревна", "HR-департамент", "Рекрутер", 70000)

        hr_director.add_subordinate(hr1)
        hr_director.add_subordinate(hr2)

        self.company.add_employee(hr1)
        self.company.add_employee(hr2)

        # Отдел продаж
```

```
        sales1 = Employee("Соколов Артем Викторович", "Отдел продаж", "Менеджер по продажам", 100000)
        sales2 = Employee("Федорова Татьяна Дмитриевна", "Отдел продаж", "Менеджер по продажам", 100000)

        sales_director.add_subordinate(sales1)
        sales_director.add_subordinate(sales2)

        self.company.add_employee(sales1)
        self.company.add_employee(sales2)

    def show_menu(self):
        """Показать меню IT-компании"""
        clear_screen()
        print_header("💻 IT-КОМПАНИЯ 💻")

        print("\n1. Показать структуру компании")
        print("2. Показать всех сотрудников")
        print("3. Найти сотрудника")
        print("4. Добавить сотрудника")
        print("5. Удалить сотрудника")
        print("6. Вернуться в главное меню")

        choice = input("\nВыберите действие: ")
        return choice

    def find_employee_ui(self):
        """Поиск сотрудника"""
        clear_screen()
        print_header("🔍 ПОИСК СОТРУДНИКА")
```

```
    name = input("Введите ФИО сотрудника: ")
    employee = self.company.find_employee(name)

    if employee:
        print(f"\nНайден сотрудник:")
        print(employee.get_info())
    else:
        print(f"\nСотрудник с ФИО '{name}' не найден.")

    wait_for_enter()

def add_employee_ui(self):
    """Добавление сотрудника"""
    clear_screen()
    print_header("➕ ДОБАВЛЕНИЕ СОТРУДНИКА")

    print("Тип сотрудника:")
    print("1. Обычный сотрудник")
    print("2. Руководитель")
    print("3. Отмена")

    emp_type = input("\nВыберите тип: ")

    if emp_type == '3':
        return

    # Ввод общих данных
    full_name = input("Введите ФИО: ")
    department = input("Введите отдел: ")
    position = input("Введите должность: ")
```

```
try:
    salary = float(input("Введите зарплату: "))
except ValueError:
    print("Ошибка: введите число!")
    wait_for_enter()
    return

# Выбор менеджера
print("\nВведите ФИО руководителя (или оставьте пустым, если руководитель не указан):")
manager_name = input()

if emp_type == '1':
    new_employee = Employee(full_name, department, position, salary)
else:
    new_employee = Manager(full_name, department, position, salary)

# Добавление в структуру
if manager_name:
    manager = self.company.find_employee(manager_name)
    if manager and isinstance(manager, Manager):
        manager.add_subordinate(new_employee)
    else:
        print("Руководитель не найден или не является менеджером!")
        wait_for_enter()
        return
elif self.company.ceo:
    self.company.ceo.add_subordinate(new_employee)

self.company.add_employee(new_employee)
```

```
    print(f"\nСотрудник {full_name} успешно добавлен!")
    wait_for_enter()

def remove_employee_ui(self):
    """Удаление сотрудника"""
    clear_screen()
    print_header("➖ УДАЛЕНИЕ СОТРУДНИКА")

    name = input("Введите ФИО сотрудника для удаления: ")
    employee = self.company.find_employee(name)

    if not employee:
        print(f"\nСотрудник с ФИО '{name}' не найден.")
        wait_for_enter()
        return

    print(f"\nНайден сотрудник:")
    print(employee.get_info())
    print()

    confirm = input("Вы уверены, что хотите удалить этого сотрудника? (д/н): ")
    if confirm.lower() in ['д', 'да', 'y', 'yes']:
        self.company.remove_employee(employee)
        print("Сотрудник удален!")
    else:
        print("Удаление отменено.")

    wait_for_enter()

def run(self):
```

```
        """Запуск приложения IT-компании"""
        while True:
            choice = self.show_menu()

            if choice == '1':
                clear_screen()
                print(self.company.get_company_structure())
                wait_for_enter()
            elif choice == '2':
                clear_screen()
                print(self.company.get_all_employees_list())
                wait_for_enter()
            elif choice == '3':
                self.find_employee_ui()
            elif choice == '4':
                self.add_employee_ui()
            elif choice == '5':
                self.remove_employee_ui()
            elif choice == '6':
                break
            else:
                print("\nНеверный выбор!")
                wait_for_enter()


# ==================== ГЛАВНОЕ МЕНЮ ====================

class MainApp:
    def __init__(self):
        self.burger_app = BurgerApp()
```

```
    self.it_company_app = ITCompanyApp()

def show_main_menu(self):
    """Показать главное меню"""
    clear_screen()
    print_header="""
╔═══════════════════════════════════════════════════════╗
║    ЛАБОРАТОРНЫЕ РАБОТЫ ПО ПАТТЕРНАМ          ║
║       ПРОЕКТИРОВАНИЯ                          ║
╚═══════════════════════════════════════════════════════╝
"""

    print("\nДоступные лабораторные работы:\n")
    print_menu_item(1, "🍔 Бургер-закусочная (Порождающий паттерн - Builder)")
    print_menu_item(2, "💻 IT-компания (Структурный паттерн - Composite)")
    print_menu_item(0, "🚪 Выход")

    choice = input("\nВыберите лабораторную работу: ")
    return choice

def run(self):
    """Запуск главного приложения"""
    while True:
        choice = self.show_main_menu()

        if choice == '1':
            self.burger_app.run()
        elif choice == '2':
            self.it_company_app.run()
        elif choice == '0':
```

```
            clear_screen()
            print_header("ДО СВИДАНИЯ!")
            print("\nСпасибо за использование программы!")
            time.sleep(2)
            sys.exit(0)
        else:
            print("\nНеверный выбор!")
            wait_for_enter()


# ==================== ТОЧКА ВХОДА ====================

if __name__ == "__main__":
    app = MainApp()
    app.run()
```

Вывод : я изучил тему и сделал задания.